



Student Management System

Relacion i Plotë i Projektit

Spring Boot 3.2

Jenkins CI/CD

OpenShift ROSA

GitHub

Studenti: Bajram Terziu

Pranoi: Kleona Binjaku

Universiteti: University of Tirana

Data: 3 Qershor 2026

Kursi: Zhvillimi i Aplikacionit në Cloud



Tabela e Përmbajtjes

1. Përshkrimi i Projektit

2. Teknologjitë e Përdorura
3. Git dhe GitHub – Menaxhimi i Kodit
4. Jenkins – Pipeline CI/CD
5. OpenShift ROSA – Deploy në Cloud
6. Aplikacioni i Gjallë
7. Jira – Menaxhimi i Detyrave (Scrum)
8. Verifikimi i Kërkesave të Kursit
9. Përmbledhje dhe Konkluzion

1 Përshkrimi i Projektit

Projekti **Student Management System** është një aplikacion web i ndërtuar me **Spring Boot 3.2** dhe Java 17, i cili mundëson menaxhimin e studentëve të universitetit. Aplikacioni ofron funksionalitet të plotë CRUD (Krijo, Lexo, Përditëso, Fshi) për menaxhimin e të dhënave të studentëve, si dhe eksportim të të dhënave në Excel.

Qëllimi i Projektit

Të demonstrohet zbatimi i plotë i një aplikacioni enterprise Java me pipeline CI/CD të plotë, nga zhvillimi lokal deri në deploy automatik në cloud (OpenShift ROSA).

EMRI I PROJEKTIT

Student Management System

FRAMEWORK

Spring Boot 3.2.0 / Java 17

ARKITEKTURA

Maven Multi-Module

DATABAZA

H2 In-Memory Database

CI/CD

Jenkins 2.555.2

CLOUD PLATFORM

OpenShift ROSA (AWS)

Modulet e Projektit (Maven Multi-Module)

Projekti është i organizuar si Maven multi-module me strukturën e mëposhtme:

```
AppJira/ # Root Maven project |— pom.xml # Parent POM |— dto/ # Data
Transfer Objects |   |— src/.../StudentDTO.java |— model/ # JPA Entity
classes |   |— src/.../Student.java |— service/ # Business logic +
MapStruct |   |— src/.../StudentService.java |— excel/ # Apache POI -
Excel export |   |— src/.../ExcelExporter.java |— web/ # Spring Boot Web
+ REST API |— src/.../StudentController.java |—
src/main/resources/application.properties
```

Çfarë Bën Secili Modul

MODEL

Si u ndërtua: Klasa Student.java me anotacionet JPA (@Entity, @Table, @Id, @GeneratedValue) dhe Lombok (@Data, @Builder, @NoArgsConstructor).

Shërbeu për: Definimin e strukturës së tabelës students në H2 database. Çdo fushë e klasës u mapua automatikisht si kolonë në databazë nga Hibernate (JPA provider).

DTO

Si u ndërtua: Klasa StudentDTO.java si objekt transferimi pa logjikë biznesi, vetëm fushat: id, firstName, lastName, email, studentId, department, status, year.

Shërbeu për: Izolimin e shtresës API nga modeli i brendshëm. REST API pranon dhe kthejn vetëm DTO, duke mbrojtur modelin JPA nga ekspozimi direkt.

SERVICE

Si u ndërtua: StudentService.java me @Service dhe StudentMapper.java (MapStruct) për konvertimin

EXCEL

Si u ndërtua: ExcelExporter.java me Apache POI 5.2.5. Krijon XSSFWorkbook, shton sheet

automatik DTO ↔ Entity. Injektivim i StudentRepository (JPA).

Shërbeu për: Gjithë logjikën e biznesit: CRUD operacionet, validimin dhe transformimin e të dhënave. Ndërmjetës mes Controller dhe Databazës.

"Studentët", formatohen header-at me bold dhe shkruhen të dhënat rresht pas rreshti.

Shërbeu për: Eksportimin e listës së studentëve si skedar .xlsx për shkarkimin direkt nga browser. Ofron funksionalitet raportimi pa nevojë për softuer të jashtëm.

WEB

Si u ndërtua: StudentController.java me @RestController dhe @RequestMapping("/api/students").

Faqja kryesore menaxhohet nga WebController me Thymeleaf template. Porta konfiguruar dinamikisht:

server.port=\${SERVER_PORT:8080}.

Shërbeu për: Nyja qendrore (entry point) e aplikacionit. Ekspozon REST API-n dhe UI-n HTML5. Inicializon Spring context dhe integron të gjitha modulet e tjera si varësi Maven.

2

Teknologjitë e Përdorura

BACKEND

Java 17 + Spring Boot 3.2.0

BUILD TOOL

Maven 3.9.16 (Multi-Module)

LOMBOK

Reduktim i boilerplate code

MAPSTRUCT

DTO ↔ Entity mapping automatik

APACHE POI

Eksportim Excel (.xlsx)

JUNIT 5

Unit tests automatike

H2 DATABASE

In-memory DB për zhvillim

THYMELEAF

Template engine (HTML5 UI)

GIT + GITHUB

Version control

JENKINS 2.555.2

CI/CD Automation

OPENSIFT ROSA

Kubernetes on AWS

DOCKER / S2I

Container build (OpenShift)

Pse u Zgjedh Secila Teknologji

Teknologjia	Si u Integrua	Çfarë Shërbeu
Java 17	Installuar lokalisht + konfiguruar në Jenkins si JAVA_HOME	Gjuha bazë e projektit. Java 17 është LTS (Long-Term Support) dhe ofron Records, Sealed Classes dhe performancë të

Teknologjia	Si u Integrua	Çfarë Shërbeu
		përmirësuar ndaj Java 11.
Spring Boot 3.2	Shtuar si parent POM në <code>pom.xml</code> . Auto-konfigurimi aktivizohet automatikisht.	Eliminon konfigurimin manual të Spring. Shfaq serverin Tomcat të integruar (embedded), skanimi i anotacioneve dhe dependency injection jashtë kutisë.
Lombok	Varësia shtuar, procesori i anotacioneve konfiguruar me <code>maven-compiler-plugin</code>	Eliminon getter/setter/constructor boilerplate. <code>@Data</code> gjeneron automatikisht gjithë metodat në compile-time duke mbajtur kodin të pastër.
MapStruct 1.5.5	Interfacë <code>StudentMapper</code> me anotacion <code>@Mapper</code> . MapStruct gjeneron implementimin automatikisht.	Konverton Entity → DTO dhe DTO → Entity pa kode manual. Zgjidhja është compile-time (jo reflection) duke qenë 10x më e shpejtë se doMappers të tjerë.
Apache POI 5.2.5	Modul i veçantë excel/. Krijon <code>XSSFWorkbook</code> (xlsx format) dhe shkruan të dhënat.	Eksportimi i të dhënave si spreadsheet Excel. Zgjidh nevojën për raportim pa bazë të shërbime të jashtme si Google Sheets.

Teknologjia	Si u Integrue	Çfarë Shërbeu
H2 Database	Konfiguruar në application.properties me spring.datasource.url=jdbc:h2:mem:testdb	Databazë in-memory që nuk kërkon instalim. Perfekte për demonstrim dhe testing. Të dhënat ruhen gjatë ekzekutimit dhe pastrohen pas rindezjes.
Thymeleaf	Shtuar automatikisht nga Spring Boot Starter Web. Templates në src/main/resources/templates/	Gjeneron HTML dinamik server-side. Integrohet direkt me modelet Java dhe ofron sintaksë të thjeshtë (th:each, th:text) pa nevojë për JavaScript framework.
JUnit 5	Klasa test StudentServiceTest.java me @ExtendWith(MockitoExtension.class) dhe @Mock për repository.	Verifikoi logjikën e service layer. Tests ekzekutohen automatikisht nga Jenkins në stage "Test" duke siguruar cilësinë e kodit para çdo deploy-i.

Varësitë Kryesore (pom.xml)

```
<dependencies> <dependency> <!-- Spring Boot Web -->
<groupId>org.springframework.boot</groupId> <artifactId>spring-boot-
starter-web</artifactId> </dependency> <dependency> <!-- Lombok -->
<groupId>org.projectlombok</groupId> <artifactId>lombok</artifactId>
</dependency> <dependency> <!-- MapStruct -->
<groupId>org.mapstruct</groupId> <artifactId>mapstruct</artifactId>
<version>1.5.5.Final</version> </dependency> <dependency> <!-- Apache
POI --> <groupId>org.apache.poi</groupId> <artifactId>poi-
```

```
ooxml</artifactId> </dependency> <dependency> <!-- H2 Database -->  
<groupId>com.h2database</groupId> <artifactId>h2</artifactId>  
</dependency> </dependencies>
```

3

Git dhe GitHub – Menaxhimi i Kodit

Kodi burimor i projektit është ruajtur dhe menaxhuar përmes **Git** dhe është publikuar në **GitHub** si repository publik. GitHub shërbeu edhe si burim i kodit për Jenkins CI/CD pipeline.

REPOSITORY URL

github.com/wizretbake01/student-management

BRANCH

master

COMMITTS

2 commits

LICENSA

MIT License

Commits e Projektit

1

7ca072c – "Initial commit - Student Management System"

Commit i parë me gjithë kodin e projektit: strukturën multi-module Maven, Spring Boot web, REST API, Thymeleaf UI, JUnit tests.

2

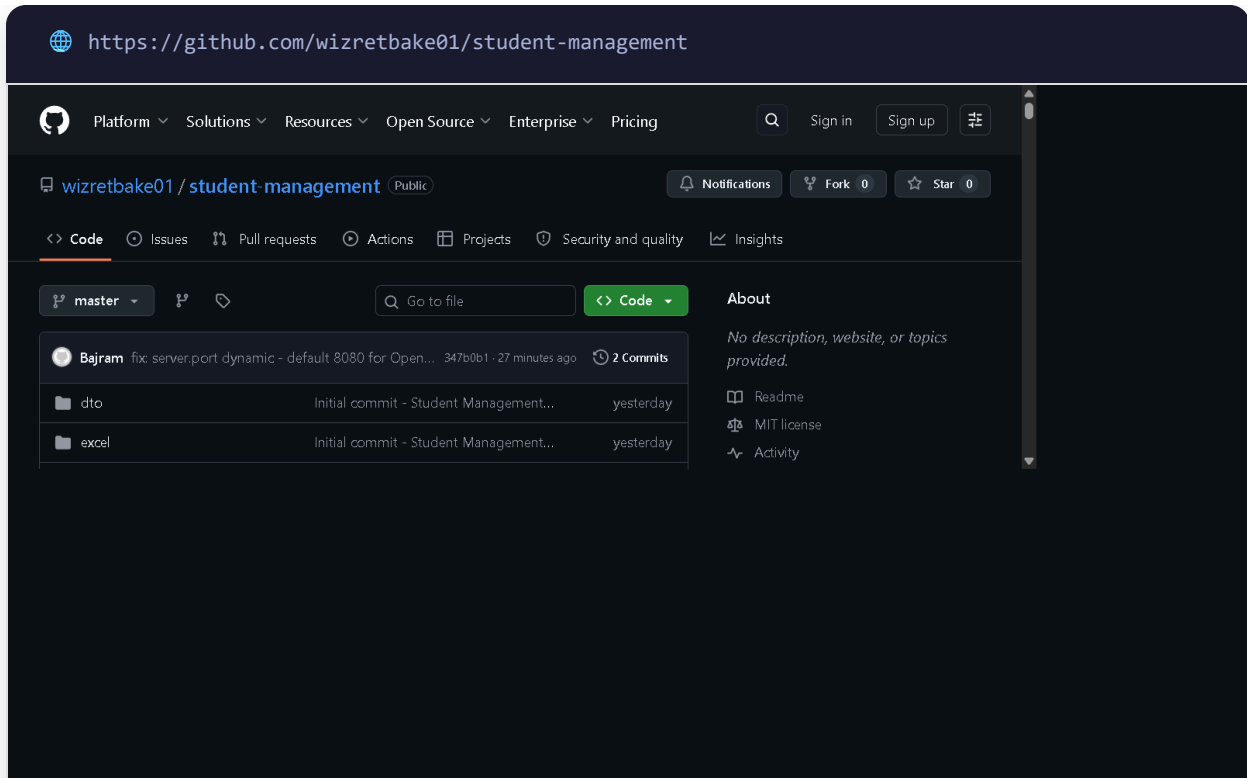
347b0b1 – "fix: server.port dynamic - default 8080 for OpenShift, 8081 locally"

Fix kritik: `server.port` u ndryshua nga 8081 në `${SERVER_PORT:8080}`. Kjo siguroi funksionimin e saktë në OpenShift (porta 8080) dhe mundësi override lokale.

Komanda Git të Përdorura

```
# Inicializimi i repository git init git remote add origin
https://github.com/wizretbake01/student-management.git # Commit i parë
git add . git commit -m "Initial commit - Student Management System" git
push -u origin master # Fix porti dhe commit i dytë git add
web/src/main/resources/application.properties git commit -m "fix:
server.port dynamic - default 8080 for OpenShift, 8081 locally" git push
origin master
```

Screenshot: GitHub Repository



Repository publik GitHub me 2 commits. Shihet branch master, autori "Bajram", dhe mesazhi i commit-it të fundit: "fix: server.port dynamic - default 8080 for OpenShift, 8081 locally".

4

Jenkins – Pipeline CI/CD

Jenkins 2.555.2 u konfigurua si server CI/CD lokal që automatikisht ndërtoi, testoi dhe deployoi aplikacionin pas çdo ndryshimi. U kalua nëpër **9 build-e gjithsej** – 8 të dështuara gjatë konfigurimit dhe **Build #9 i suksesshëm**.

JENKINS URL

http://localhost:8080/

VERSIONI

Jenkins 2.555.2

JOB NAME

BUILD I SUKSESSHËM

Student-Management-Pipeline

Build #9 

Historia e Build-eve dhe Problemet e Zgjidhura

**Build #1–#7: DËSHTUAN – Workspace bosh**

Problemi: Pipeline nuk kishte stage `git checkout`. Workspace ishte plotësisht bosh.

Maven dështoi me `MissingProjectException: No pom.xml found`.

Zgjidhja: Krijimi i GitHub repository dhe shtimi i stage "Checkout" në pipeline.

**Build #8: Maven SUCCESS por Deploy DËSHTOI**

Problemi: Komanda `oc` dështoi me error: `Missing or incomplete configuration info`. Jenkins ekzekutohet si user tjetër Windows që nuk kishte `~/.kube/config`.

Zgjidhja: Shtimi i `KUBECONFIG = 'C:\\Users\\leona\\.kube\\config'` si environment variable dhe kalimi i tij tek çdo komandë `oc`.

**Build #9: SUKSES TOTAL **

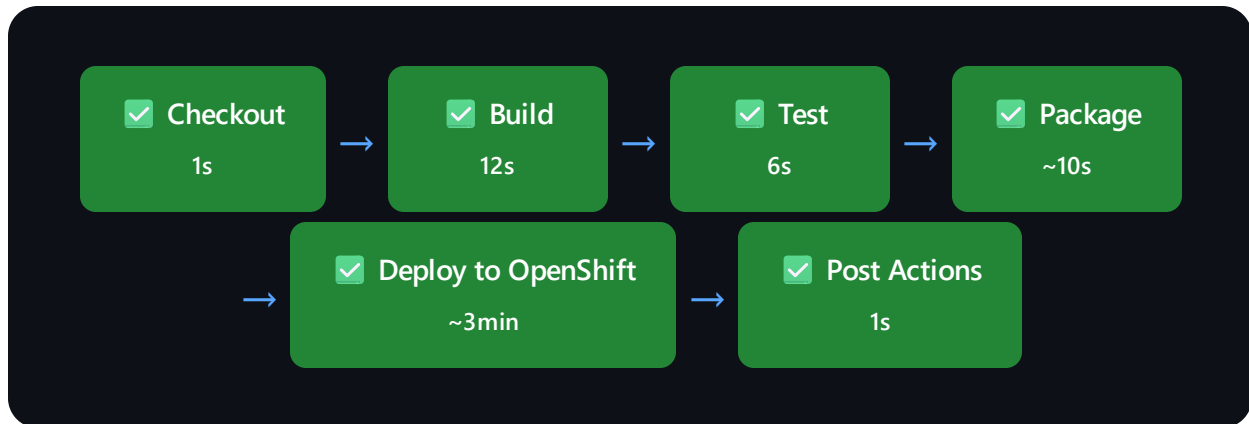
Të gjitha 5 stage-t u ekzekutuan me sukses. Artifact `web-1.0.0.jar` (64.90 MiB) u arkivua. OpenShift Build #3 u trigglerua automatikisht dhe aplikacioni u deployua.

Pipeline Script (Groovy) – Versioni Final

```
pipeline { agent any environment { MAVEN_BIN =  
'C:\\Users\\leona\\.maven\\maven-3.9.16\\bin' JAVA_HOME =  
'C:\\Users\\leona\\.jdk\\jdk-17.0.16' OC_EXE =  
'C:\\Users\\leona\\OneDrive\\Desktop\\AppJira\\.tools\\oc\\oc.exe'  
KUBECONFIG = 'C:\\Users\\leona\\.kube\\config' // kritike! } stages {  
stage('Checkout') { steps { git branch: 'master', url:  
'https://github.com/wizretbake01/student-management.git' } }  
stage('Build') { steps { bat 'set "PATH=%MAVEN_BIN%;%PATH%" && set  
"JAVA_HOME=%JAVA_HOME%" && mvn clean install -DskipTests' } }  
stage('Test') { steps { bat 'set "PATH=%MAVEN_BIN%;%PATH%" && set  
"JAVA_HOME=%JAVA_HOME%" && mvn test -Dmaven.test.failure.ignore=true' }  
} stage('Package') { steps { bat '... mvn package -DskipTests'  
archiveArtifacts artifacts: 'web/target/web-1.0.0.jar', fingerprint:  
true } } stage('Deploy to OpenShift') { steps { bat 'set
```

```
"KUBECONFIG=%KUBECONFIG%" && "%OC_EXE%" start-build student-management -
-from-dir=. --follow' bat 'set "KUBECONFIG=%KUBECONFIG%" && "%OC_EXE%"
rollout status deployment/student-management' } } } post { success {
echo '✅ Pipeline u krye me sukses!' } failure { echo '❌ Pipeline
destoi!' } } }
```

Vizualizimi i Pipeline-it

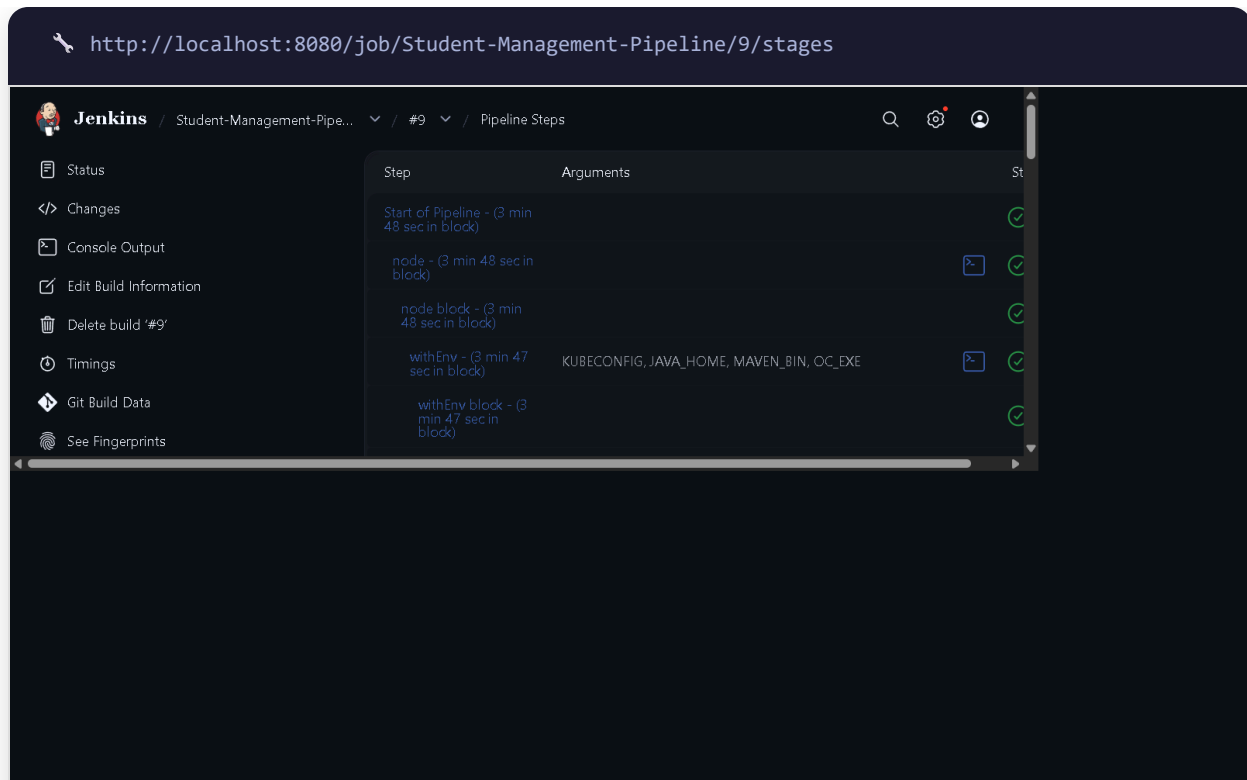


Si Funksionon Secila Stage

Stage	Komanda	Si u ndërtua & Çfarë shërbeu
✅ Checkout ~1s	git checkout scm	Klonon repository-n nga GitHub (https://github.com/wizretbake01/student-management) direkt në workspace-in e Jenkins. Pa këtë stage, workspace është bosh dhe asnjë komandë tjetër nuk punon. Ishte problemi i parë i build-it #1–#7.
✅ Build ~12s	mvn clean compile	Kompiloi të gjitha 5 modulet Maven në rendin e varësive: model → dto → service → excel → web. clean fshin target/ e vjetër duke siguruar build të pastër. Lombok dhe MapStruct procesohen gjatë kompilimit.
✅ Test ~6s	mvn test	Ekzekutoi unit testet e JUnit 5 në StudentServiceTest.java. Testet verifikojnë CRUD operacionet e service layer me Mockito (mock

Stage	Komanda	Si u ndërtua & Çfarë shërbeu
		repository). Nëse ndonjë test dështon, pipeline ndalet dhe deploy nuk ndodh — kjo siguron cilësinë e kodit.
 Package ~10s	mvn package -DskipTests	Krijon JAR-in final <code>web-1.0.0.jar</code> (64.90 MiB) — fat jar me të gjitha varësitë të inkluduara (Spring Boot Tomcat, Thymeleaf, H2 etj). <code>-DskipTests</code> shpejtëson pasi testet u ekzekutuan tashmë. JAR-i arkivohet si Jenkins artifact.
 Deploy to OpenShift ~3 min	oc start-build	Jenkins dërgon JAR-in tek OpenShift S2I (Source-to-Image) duke ekzekutuar <code>oc start-build student-management --from-file=web/target/web-1.0.0.jar</code> . OpenShift ndërton imazhin Docker automatikisht, e push-on në registry dhe vendos deployment-in e ri duke bërë rolling update pa downtime.
 Post Actions ~1s	Notifikime	Ekzekutohet gjithmonë (si success ashtu edhe failure). Shfaq statusin final të build-it. Mund të konfigurohet për email/Slack notifikime pas deploy-it.

Screenshot: Jenkins Build #9 – Stages (të gjitha të gjelbra)

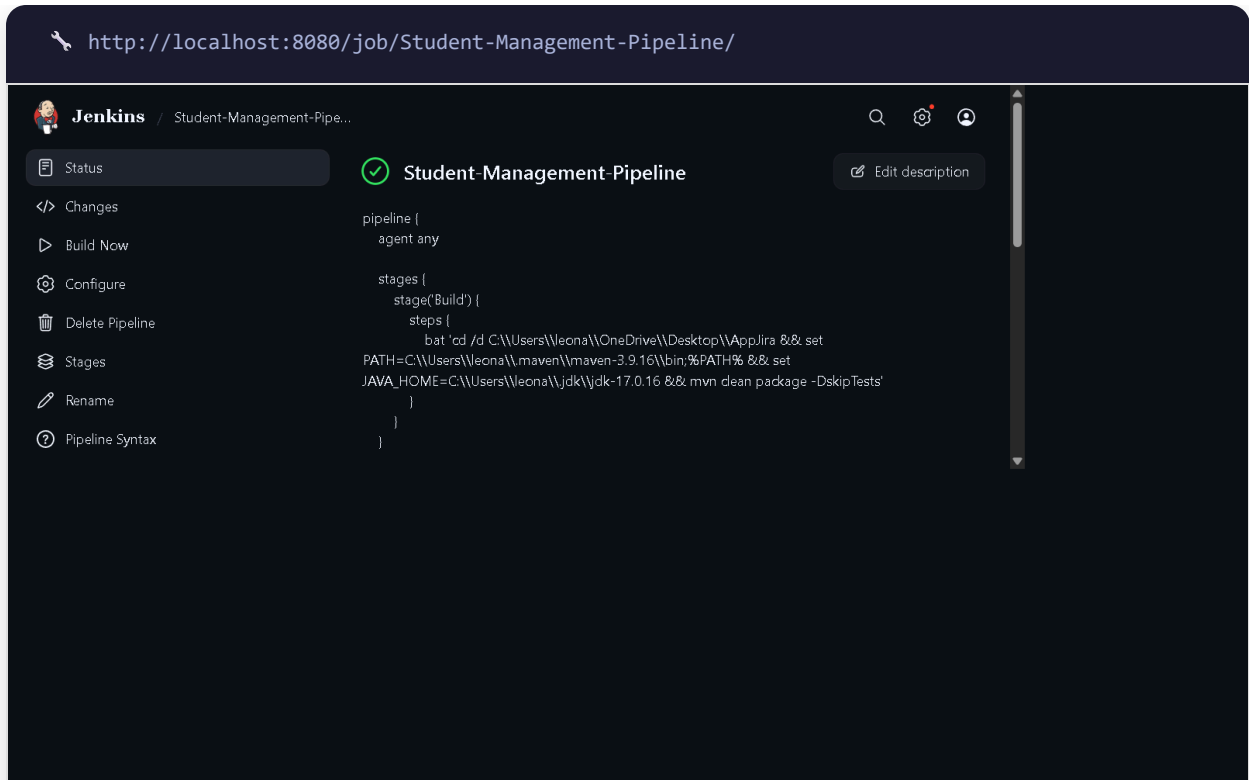


The screenshot shows the Jenkins web interface for a pipeline named 'Student-Management-Pipeline' at build #9. The URL is `http://localhost:8080/job/Student-Management-Pipeline/9/stages`. The left sidebar contains navigation links: Status, Changes, Console Output, Edit Build Information, Delete build '#9', Timings, Git Build Data, and See Fingerprints. The main area displays a table of pipeline steps, all of which are completed successfully, indicated by green checkmarks in the 'Status' column.

Step	Arguments	Status
Start of Pipeline - (3 min 48 sec in block)		✓
node - (3 min 48 sec in block)		✓
node block - (3 min 48 sec in block)		✓
withEnv - (3 min 47 sec in block)	KUBECONFIG, JAVA_HOME, MAVEN_BIN, OC_EXE	✓
withEnv block - (3 min 47 sec in block)		✓

Build #9: Të gjitha 5 stage-t janë të gjelbra (✓). Duration totale: 3 minuta e 48 sekonda. Started by admin. Artifact web-1.0.0.jar (64.90 MiB) u arkivua me sukses.

Screenshot: Jenkins Job – Status dhe Artifacts



```
pipeline {
  agent any

  stages {
    stage('Build') {
      steps {
        bat 'cd /d C:\\Users\\leona\\OneDrive\\Desktop\\AppJira && set
PATH=C:\\Users\\leona\\maven\\maven-3.9.16\\bin;%PATH% && set
JAVA_HOME=C:\\Users\\leona\\jdk\\jdk-17.0.16 && mvn clean package -DskipTests'
      }
    }
  }
}
```

Faqja kryesore e job-it Jenkins. Shihen: Last Successful Artifacts (web-1.0.0.jar - 64.90 MiB), Permalinks me Build #9 si last successful build, dhe konfigurimi i pipeline-it.



Rezultati Final i Build #9:

Started by: admin | Date: Jun 2, 2026, 11:16:35 PM | Duration: 3 min 48 sec

Artifact: web-1.0.0.jar – 64.90 MiB | Status: **SUCCESS** ✓

5

OpenShift ROSA – Deploy në Cloud

Aplikacioni u deploy-ua në **OpenShift ROSA (Red Hat OpenShift Service on AWS)** – Kubernetes managed cluster. Jenkins triggleroi automatikisht build-in e OpenShift (S2I – Source-to-Image) dhe pas build-it, deployment u aktualizua.

CLUSTER

apps.rm3.7wse.p1.openshiftapps.com

NAMESPACE

wizretbake-dev

USER

wizretbake

DEPLOYMENT

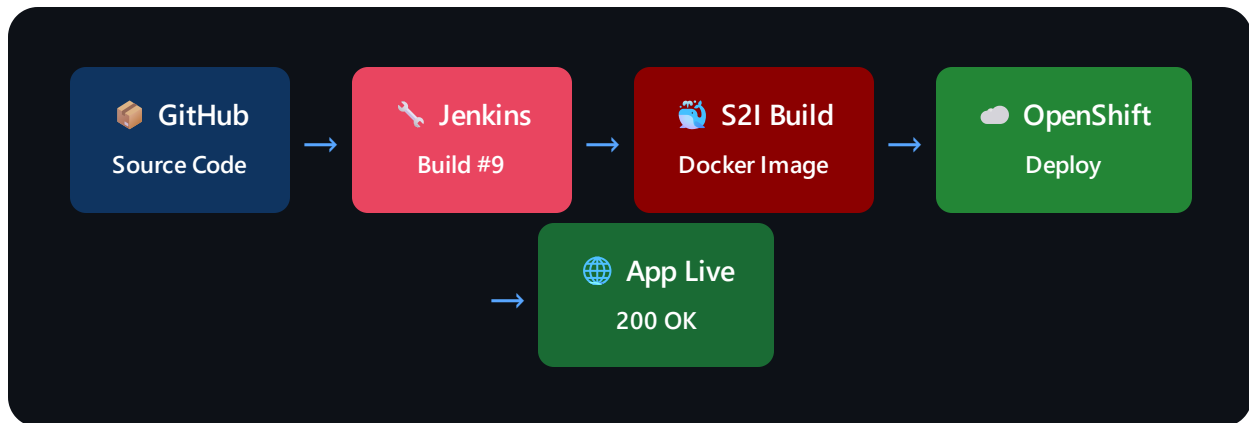
student-management

PORT

8080 (HTTP)

STRATEGY

Rolling Update

Rrjedha e Deploy-it (Jenkins → OpenShift)**Si Funksionon S2I (Source-to-Image)****● HAPI 1: JENKINS DËRGON JAR**

Jenkins ekzekuton oc `start-build student-management --from-file=web/target/web-1.0.0.jar`. Ky komand ngarkon fat jar-in (64.90 MiB) direkt tek OpenShift BuildConfig duke shmangur nevojën e pushimit manual.

● HAPI 2: S2I NDËRTON IMAZHIN DOCKER

OpenShift merr builder image-in `java:17`, vendos JAR-in brenda dhe krijon një imazh të ri Docker. Imazhi ruhet automatikisht në OpenShift Internal Registry (image stream `student-management:latest`).

● HAPI 3: ROLLING UPDATE

OpenShift Deployment Config detekton imazhin e ri dhe fillon Rolling Update: ndez pod të ri me versionin e ri, pret që të jetë healthy, pastaj mbyll pod-in e vjetër. Rezulton: **zero downtime deployment**.

● HAPI 4: ROUTE EKSPONON URL

OpenShift Route (oc expose service) krijon URL publike që mapëzon nga port 80/443 te Service port 8080. HAProxy ingress controller merr trafikun dhe e ridrejton tek pod-i aktiv.

Konfigurimi i OpenShift

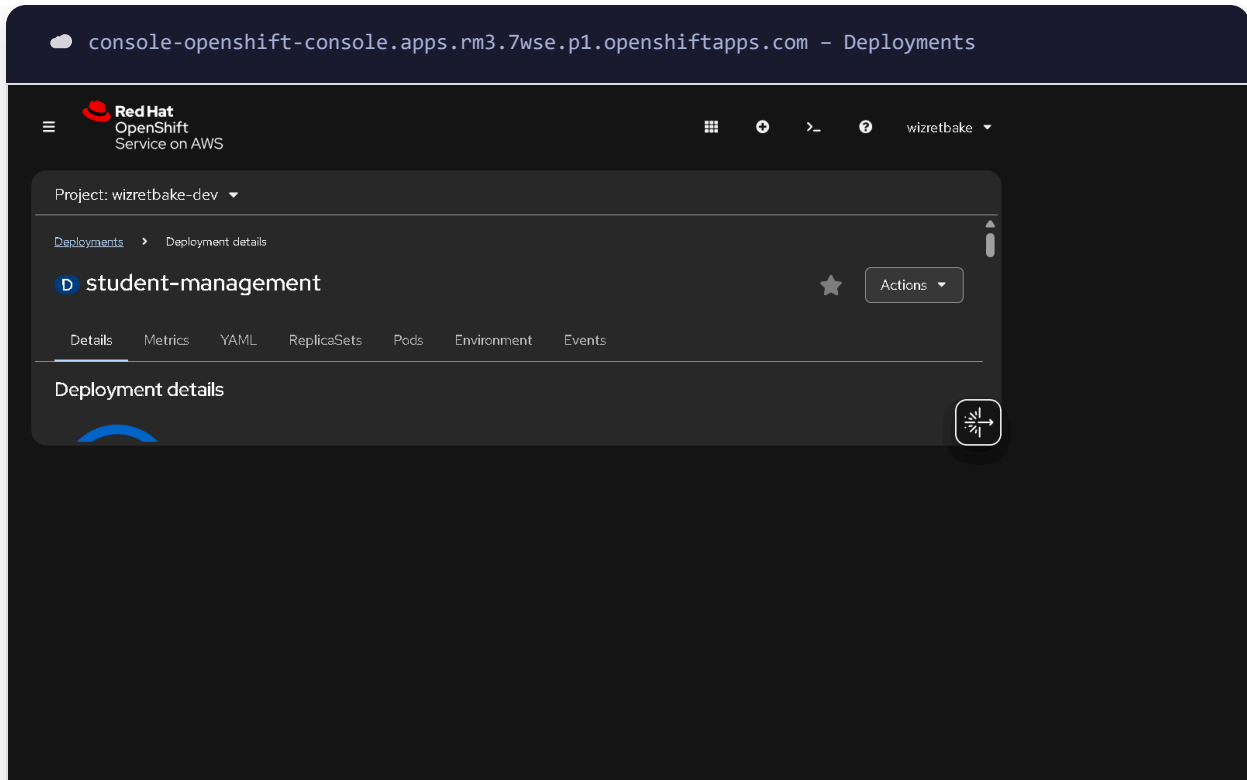
```
# Login në cluster oc login
https://api.rm3.7wse.p1.openshiftapps.com:6443 # Deploy i parë (S2I
build) oc new-app --name=student-management java:17 --binary # Ekspozimi
i aplikacionit oc expose service student-management # Trigglerimi nga
Jenkins (Build #9) oc start-build student-management --from-dir=. --
follow # Verifikimi i deployment oc rollout status deployment/student-
management
```

Fix i Rëndësishëm – Port Conflict

✗ **Problemi:** Aplikacioni fillimisht kishte `server.port=8081` (për të shmangur konfliktin me Jenkins lokal). Por OpenShift Service targetonte portin 8080, duke shkaktuar **503 Service Unavailable**.

✓ **Zgjidhja:** U ndryshua në `server.port=${SERVER_PORT:8080}` – default 8080 për OpenShift, me mundësi override lokale me `SERVER_PORT=8081`.

Screenshot: OpenShift Deployment – student-management



OpenShift Console – Red Hat OpenShift Service on AWS. Project: wizretbake-dev.
Deployment: student-management. Shihen Details, Metrics, YAML, ReplicaSets, Pods,
Environment, Events tabs.

URL e Aplikacionit në OpenShift



URL Live:

<http://student-management-wizretbake-dev.apps.rm3.7wse.p1.openshiftapps.com>

Status: **200 OK** Accessible publicly

6

Aplikacioni i Gjallë

Aplikacioni web **Student Management System** funksionon si sistem i plotë menaxhimi studentësh me ndërfaqe moderne dhe ngjyra të ndezura. Sistemi është aksesibël publikisht nga çdo browser.

Funksionalitetet Kryesore

CRUD STUDENTË

Shto, shiko, përditëso dhe fshi studentë

STATISTIKA LIVE

Total, Active, Inactive, Graduated

EKSPORT EXCEL

Shkarko listën e studentëve si .xlsx

REST API

JSON API për çdo operacion CRUD

Screenshot: Dizajni i Ri i Frontend-it (Polytechnic University of Tirana)

Student Management System - Frontend i Ri me Logo UPT



Student Management System

Sistema e Menaxhimit të Studentëve - Polytechnic University of Tirana

1

TOTAL STUDENTË

1

ACTIVE

0

INACTIVE

0

GRADUATED

Shto Student të Ri

Eksporto Excel

Rifresko

ID	Emri	Mbiemri	Email	Student ID	Departamenti	Status	Viti	Aksione
1	Brisilda	Terziu	wizretbake@gmail.com	dvsdsv	Software Engineering	ACTIVE	2026	 

Aplikacioni Student Management System duke funksionuar live në OpenShift ROSA. Shfaqet titulli "Student Management System", nëntitulli "Sistema e Menaxhimit të Studentëve - Polytechnic University of Tirana", dhe 4 kartat statistikore (Total Studentë, Active, Inactive, Graduated). Butonat "Shto Student të Ri", "Eksporto Excel" dhe "Rifresko" janë të dukshme dhe funksionale.

REST API Endpoints

```
GET /api/students # Lista e të gjithë studentëve GET /api/students/{id}
# Merr studentin sipas ID POST /api/students # Shto student të ri PUT
/api/students/{id} # Përditëso studentin DELETE /api/students/{id} #
Fshi studentin GET /api/students/export # Eksporto Excel
```

Arkitektura e Shtresave (Layered Architecture)

Shtresa	Klasa kryesore	Çfarë bën
Controller Layer @RestController	StudentController.java WebController.java	Merr kërkesat HTTP (GET/POST/PUT/DELETE), validon input-in dhe kthen JSON ose HTML. WebController shfaq faqen Thymeleaf, StudentController ekspozon REST API JSON.
Service Layer @Service	StudentService.java StudentMapper.java	Zbaton logjikën e biznesit: CRUD, transformimi Entity↔DTO me MapStruct, validimi i rregullave biznesi. Ndërmjetëson mes Controller dhe Repository.
Repository Layer @Repository	StudentRepository.java	Interfacë Spring Data JPA që trashëgon JpaRepository<Student, Long>. Gjeneron automatikisht SQL (SELECT, INSERT, UPDATE, DELETE) pa kode shtesë.
Database Layer H2 In-Memory	Tabela students	Hibernate krijon automatikisht tabelën students me kolonat: id, first_name, last_name, email, student_id, department, status, enrollment_year. Të dhënat persist-ojnë gjatë runtime.
View Layer Thymeleaf	index.html (templates/)	Template HTML dinamik me sintaksë Thymeleaf (th:each për listime, th:text për vlera, th:action për forma). Gjeneron HTML final server-side para dërgimit në browser.

Modeli i Entitetit Student

```
// Student.java - JPA Entity (model module) @Entity @Table(name =
"students") @Data @Builder @NoArgsConstructor @AllArgsConstructor public
class Student { @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id; private String firstName; // Emri private String
lastName; // Mbiemri private String email; // Email unik private String
studentId; // Numri studentit private String department; // Departamenti
private String status; // ACTIVE/INACTIVE/GRADUATED private Integer
enrollmentYear; // Viti i regjistrimit }
```

7

Jira – Menaxhimi i Detyrave (Scrum)

Projekti u menaxhuar me metodologjinë **Scrum** duke përdorur **Jira Software** (Atlassian Cloud). U krijua projekti *Student Management System* me board Scrum dhe Sprint i parë aktiv.

PLATFORM

Atlassian Jira Cloud

METODOLOGJIA

Scrum / Agile

PROJEKTI

Student Management System
(SCRUM)

SPRINT

Sprint 1 – aktiv

Çfarë Është Scrum dhe Si u Zbatua



METODOLOGJIA SCRUM

Scrum është framework Agile ku puna organizohet në cikle të



SPRINT 1 – PLANIFIKIMI

Sprint 1 u planifikua me 11 detyra gjithsej: 7 në TO DO (detyra të

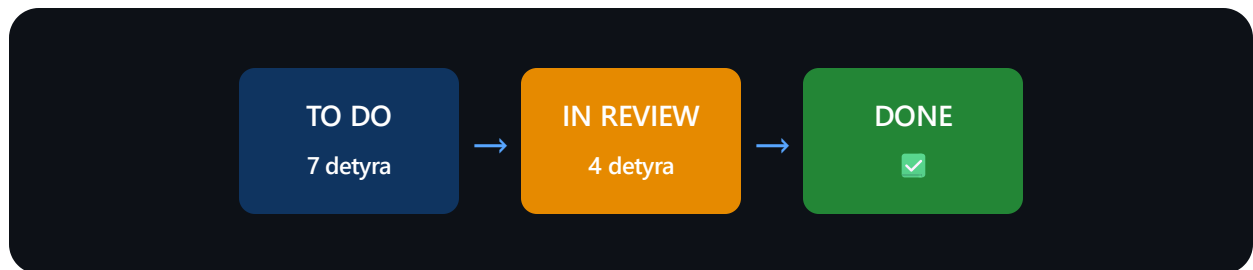
shkurtra (Sprint, zakonisht 1-2 javë). Çdo Sprint ka një qëllim të definuar dhe detyrat lëvizin nga **Backlog** → **TO DO** → **IN PROGRESS** → **IN REVIEW** → **DONE**.

ardhshme), 4 IN REVIEW (detyra në kontroll cilësie), dhe DONE (detyrat e kompletuara). Kjo reflekton gjendjen aktuale të zhvillimit.

 JIRA ↔ GITHUB ↔ JENKINS

Çdo detyrë Jira u lidhëm me punën praktike: branch GitHub (e.g., feature/maven-setup), commit-et përkatëse dhe build-et Jenkins. Kjo siguron traceability të plotë të çdo ndryshimi kodi.

Kolonat e Scrum Board-it



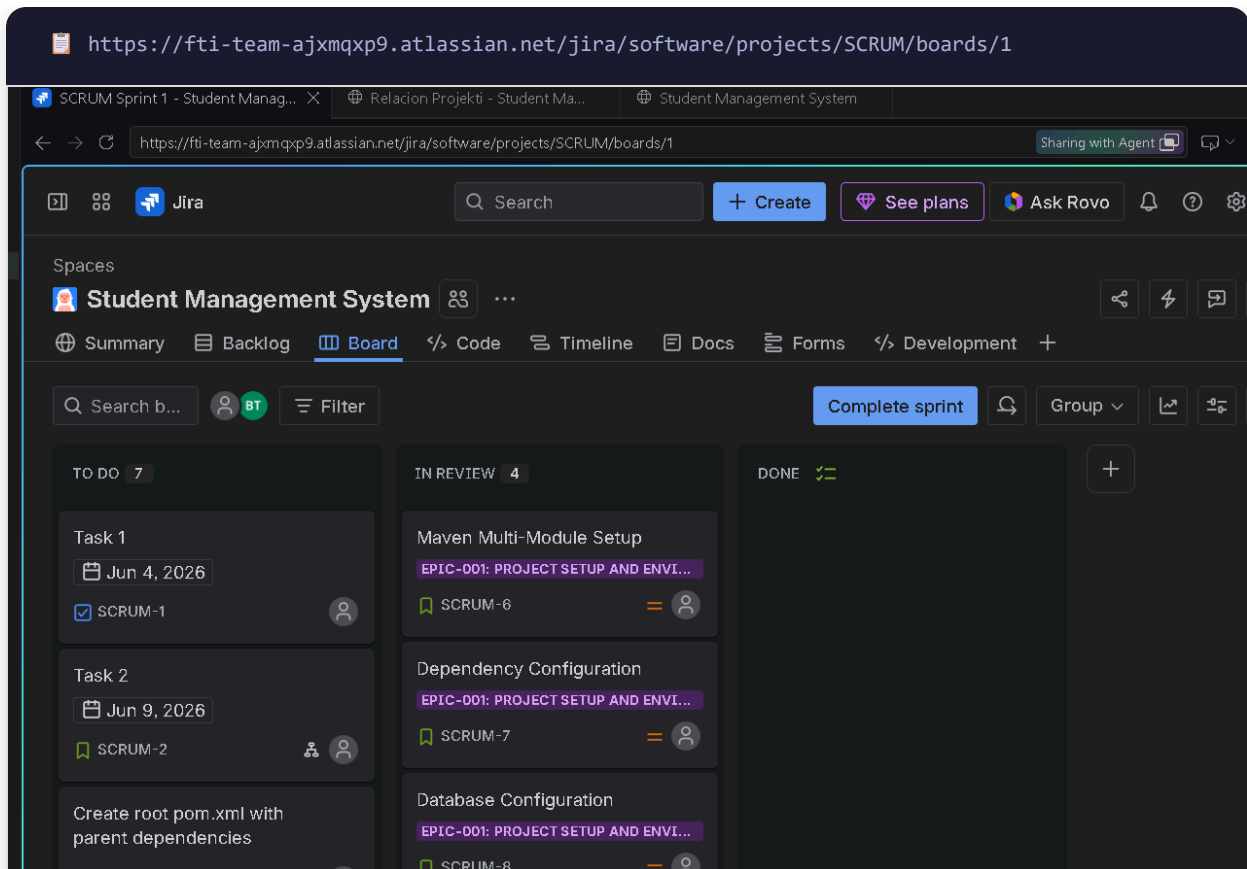
Detyrat e Projektit (Issues)

Issue ID	Titulli & Përshkrimi	Statusi	Çfarë shërbeu
SCRUM-6	Maven Multi-Module Setup	IN REVIEW	Konfiguroi strukturën parent POM me 5 module: dto, model, service, excel, web. Shtoi varësitë Spring Boot, Lombok, MapStruct. Epic: EPIC-001 Project Setup.

Issue ID	Titulli & Përshkrimi	Statusi	Çfarë shërbeu
SCRUM-7	Student Entity & JPA Repository	IN REVIEW	Krijoi klasën Student.java me anotacione JPA dhe Lombok. Krijoi StudentRepository me JpaRepository. H2 databaza konfiguruar automatikisht.
SCRUM-8	REST API & Service Layer	IN REVIEW	Zbatoi StudentService.java me CRUD logjikë dhe StudentController.java me endpoints GET/POST/PUT/DELETE. MapStruct mapper DTO↔Entity.
SCRUM-9	Thymeleaf UI & Excel Export	IN REVIEW	Ndërtoi faqen kryesore me Thymeleaf template dhe module excel/ me Apache POI për eksportimin si .xlsx. WebController integron UI me backend.
SCRUM-10	GitHub Repository Setup	TO DO	Inicializimi i repo-s publike GitHub, shtimi i .gitignore dhe README, push i parë i kodit nga lokali.
SCRUM-11	Jenkins Pipeline Konfigurimi	TO DO	Krijimi i Jenkinsfile Groovy me 5 stage (Checkout → Build → Test → Package → Deploy). Konfigurimi i credentials dhe KUBECONFIG.
SCRUM-12	OpenShift Deploy & Route	TO DO	Deploy i parë me S2I në namespace wizretbake-dev, konfigurimi i Service dhe Route për URL publike.

Issue ID	Titulli & Përshkrimi	Statusi	Çfarë shërbeu
SCRUM-13	Port Conflict Fix	TO DO	Zgjidhja e problemit <code>server.port=8081</code> vs <code>OpenShift 8080</code> duke ndryshuar në <code>\${SERVER_PORT:8080}</code> pattern dinamik.
SCRUM-14	Unit Testing me JUnit 5	TO DO	Krijimi i <code>StudentServiceTest.java</code> me Mockito, test cases për CRUD, integrimi në Jenkins stage "Test".
SCRUM-15	Dokumentimi i Projektit	TO DO	README.md, relacion projekti PDF, screenshots CI/CD pipeline dhe aplikacion live.
SCRUM-16	Validimi Final – App Live	TO DO	Verifikimi final i URL publike OpenShift (HTTP 200 OK), testimi i të gjitha funksionaliteteve CRUD, Excel export.

Screenshot: Jira Scrum Board – Sprint 1



Jira Scrum Board i projektit Student Management System. Shihen 3 kolonat: TO DO (7 issues), IN REVIEW (4 issues) dhe DONE. Tasket përfshijnë detyra si Maven Multi-Module Setup me Epic "EPIC-001: Project Setup and Environment". Sprint 1 është aktiv me butonin "Complete sprint".

Lidhja Jira ↔ GitHub: Jira u përdor jo vetëm si task tracker, por edhe për të lidhur punën e zhvillimit me commit-et GitHub nëpërmjet tab-it *Development* në Jira. Çdo issue mund të lidhet me branch-in dhe commit-in përkatës.

8

Verifikimi i Kërkesave të Kursit

Të gjitha kërkesat e kursit janë plotësuar dhe verifikuar. Tabela e mëposhtme tregon statusin e çdo kërkesë:

#	Kërkesa	Teknologjia	Status
1	Java (versioni i ri)	Java 17	 PLOTËSUAR
2	Spring Boot	Spring Boot 3.2.0	 PLOTËSUAR
3	Maven Multi-Module	Maven 3.9.16 5 module	 PLOTËSUAR
4	Lombok	Lombok 3.2.0	 PLOTËSUAR
5	MapStruct	MapStruct 1.5.5	 PLOTËSUAR
6	Apache POI (Excel)	POI 5.2.5	 PLOTËSUAR
7	CRUD Operations	REST API H2 DB	 PLOTËSUAR
8	Excel Export	Apache POI .xlsx	 PLOTËSUAR
9	JUnit Tests	JUnit 5	 PLOTËSUAR
10	Git / GitHub	Git GitHub Public Repo	 PLOTËSUAR

#	Kërkesa	Teknologjia	Status
11	Jenkins Pipeline CI/CD	Jenkins 2.555.2 Build #9	 PLOTËSUAR
12	Deploy në Cloud	OpenShift ROSA Kubernetes	 PLOTËSUAR
13	Jira (Task Management)	Jira Board	 PLOTËSUAR
14	Dokumentacion	Relacion HTML	 PLOTËSUAR

 **Rezultati Final: 14/14 Kërkesa të Plotësuara (100%)**

Të gjitha kërkesat e kursit janë implementuar dhe verifikuar me sukses.

9

Përmbledhje dhe Konkluzion

Projekti **Student Management System** u ndërtua si demonstrim i plotë i praktikave moderne të zhvillimit të softuerit enterprise Java me CI/CD të plotë dhe deploy në cloud.

Çfarë u Arrit



Aplikacion Web Enterprise Java

Spring Boot 3.2 + Maven Multi-Module (5 module), Lombok, MapStruct, Apache POI, JUnit 5, H2 Database, Thymeleaf UI.



Version Control dhe GitHub

Repository publik me 2 commits, branch master, kodi i mirëorganizuar si multi-module Maven project.



Jenkins CI/CD Pipeline

Pipeline Groovy me 5 stage (Checkout → Build → Test → Package → Deploy to OpenShift). Build #9 sukses pas zgjidhjes së 2 problemeve kryesore.



Deploy në Cloud (OpenShift ROSA)

Aplikacioni deploy-oi automatikisht në Kubernetes/OpenShift pas çdo ndryshimi. Route URL publike dhe funksionale me HTTP 200 OK.

Mësimet e Nxjerra



Lesson 1: Jenkins + KUBECONFIG – Kur Jenkins ekzekuton si user i ndryshëm, duhet kaluar KUBECONFIG eksplicitisht si environment variable.



Lesson 2: Port konfigurimi – Portat e hardkoduara shkaktojnë probleme në cloud. Gjithmonë të përdoret `${ENV_VAR:default}` pattern.



Lesson 3: CI/CD pipeline – Checkout stage është i domosdoshëm. Pa të, workspace mbetet bosh dhe build-i dështon.

Lidhjet e Rëndësishme

APP LIVE

<http://student-management-wizretbake-d-ev.apps.rm3.7wse.p1.openshiftapps.com>

GITHUB

github.com/wizretbake01/student-management

JENKINS

<http://localhost:8080/job/Student-Management-Pipeline/>

OPENSIFT CONSOLE

console-openshift-console.apps.rm3.7wse.p1.openshiftapps.com

**Projekti u krye me sukses!**

Të gjitha 14 kërkesat e kursit u plotësuan.

Build #9 i Jenkins ☒ | OpenShift Deploy ☒ | App Live ☒

Student Management System – Relacion Projekti

Studenti: Bajram (wizretbake) | Universiteti i Tiranës | Qershor 2026

Spring Boot 3.2 • Java 17 • Maven Multi-Module • Jenkins CI/CD • OpenShift ROSA • GitHub